Repaso del día 2

16 preguntas sobre hash, claves pública/privada y redes P2P

Las 16 preguntas — Repaso día 2

1. ¿Qué es una función hash y para qué sirve en blockchain?
2. ¿Qué propiedades clave tiene una función hash criptográfica?
3. ¿Por qué MyHash con P = 7 (resto de la división por 7) falla como función hash?
4. ¿Qué algoritmo de hash usa Bitcoin y cuál Ethereum?
5. ¿Qué pasó con SHA-1 en 2017?
6. ¿Qué es un Merkle Tree y para qué se usa en blockchain?
7. ¿Qué es un par de claves pública/privada, y cuál se reparte y cuál se guarda?
8. ¿Cuál es la diferencia entre cifrar y firmar?
9. ¿Qué pasa si pierdes (o filtras) tu clave privada?
10. ¿Por qué no se puede usar simplemente un hash para hacer una clave pública con la que otros cifren?
11. ¿En qué se diferencian las direcciones de Bitcoin y de Ethereum?
12. ¿Qué es una red P2P y cómo se diferencia del modelo cliente-servidor?
13. ¿Por qué Napster se vino abajo cuando cerraron su servidor?
14. ¿Qué innovación trajo BitTorrent respecto a redes P2P anteriores como Napster o eMule?
15. Cita dos ventajas y dos problemas típicos de las redes P2P.
16. ¿Por qué blockchain necesita una red P2P para funcionar?

PREGUNTA 1 / 16

Qué es un hash

Pregunta

¿Qué es una función hash y para qué sirve en blockchain?

Respuesta razonada

Es una función matemática que toma cualquier dato (un texto, un PDF, una transacción) y devuelve una huella digital de tamaño fijo. En blockchain se usa para tres cosas: garantizar la integridad de los bloques (cualquier cambio en el contenido cambia el hash), encadenar bloques (cada bloque incluye el hash del anterior), y crear identificadores únicos para transacciones, direcciones y datos.

PREGUNTA 2 / 16

Propiedades del hash

Pregunta

¿Qué propiedades clave tiene una función hash criptográfica?

Respuesta razonada

Cinco propiedades. Determinismo: el mismo input siempre produce el mismo hash. Tamaño fijo: la salida es siempre del mismo tamaño (256 bits en SHA-256), sin importar el tamaño del input. Una sola dirección: del hash no se puede recuperar el input original. Avalancha: cambiar un bit del input cambia ≈ 50% del hash. Sin colisiones prácticas: encontrar dos inputs distintos con el mismo hash es matemáticamente posible pero computacionalmente inviable.

PREGUNTA 3 / 16

Por qué falla MyHash

Pregunta

¿Por qué MyHash con P = 7 (resto de la división por 7) falla como función hash?

Respuesta razonada

Porque solo tiene 7 outputs posibles: {0, 1, 2, 3, 4, 5, 6}, frente a infinitos inputs. Por el principio del palomar, con solo 8 inputs distintos ya tienes garantizada al menos una colisión. SHA-256, en cambio, tiene 2^{256} outputs distintos: las colisiones existen matemáticamente pero encontrar una a propósito requiere más operaciones que átomos hay en el universo observable.

PREGUNTA 4 / 16

Bitcoin y Ethereum

Pregunta

¿Qué algoritmo de hash usa Bitcoin y cuál Ethereum?

Respuesta razonada

Bitcoin usa SHA-256 (publicado por la NSA en 2001, vigente). Ethereum usa Keccak-256 (la versión original previa al estándar SHA-3). Los dos producen un hash de 256 bits (32 bytes), pero son algoritmos distintos por dentro y dan resultados distintos para el mismo input. Es un buen ejemplo de que cada blockchain hace sus propias elecciones criptográficas, no hay una única correcta.

PREGUNTA 5 / 16

SHA-1 en 2017

Pregunta

¿Qué pasó con SHA-1 en 2017?

Respuesta razonada

Google y CWI publicaron el ataque shattered.io: dos PDFs distintos con exactamente el mismo SHA-1. El coste fue enorme (≈ 110 GPUs durante un año), pero demostraron que era factible. Antes de 2017 se decía que SHA-1 seguía siendo seguro; tras el ataque, todos los sistemas serios migraron a SHA-256 o superior. La lección: la criptografía caduca, lo que es seguro hoy puede no serlo dentro de unos años.

PREGUNTA 6 / 16

Merkle Tree

Pregunta

¿Qué es un Merkle Tree y para qué se usa en blockchain?

Respuesta razonada

Es un árbol binario donde cada hoja es el hash de un dato y cada nodo interno es el hash de sus dos hijos concatenados. La raíz (Merkle root) resume todo el árbol en un solo hash de 32 bytes. En Bitcoin, cada bloque incluye la Merkle root de todas sus transacciones; cambia una sola transacción y la raíz cambia. Permite demostrar que un dato pertenece al conjunto enviando solo $\log_2(N)$ hashes — para 1024 transacciones, 10 hashes bastan.

PREGUNTA 7 / 16

Pública y privada

Pregunta

¿Qué es un par de claves pública/privada, y cuál se reparte y cuál se guarda?

Respuesta razonada

Son dos números matemáticamente ligados. La privada se mantiene en secreto y de ella deriva la pública, que sí se reparte libremente. Cumplen una asimetría fundamental: con la pública cualquiera puede hacer operaciones que solo la privada puede deshacer (y viceversa). Si alguien tiene tu privada, puede actuar como tú; por eso protegerla es lo más crítico. La pública se publica sin riesgo, la privada jamás sale de tu poder.

PREGUNTA 8 / 16

Cifrar vs firmar

Pregunta

¿Cuál es la diferencia entre cifrar y firmar?

Respuesta razonada

Son dos operaciones distintas con el mismo par de claves. Cifrar: cualquiera con tu clave pública mete un mensaje 'en una caja' que solo tu privada puede abrir → confidencialidad. Firmar: tú con tu privada generas un código único sobre un mensaje que cualquiera con tu pública puede verificar → autenticidad (fui yo) e integridad (nadie cambió el mensaje). En blockchain se usa sobre todo firmar — las transacciones no se cifran porque son públicas, pero sí se firman para demostrar quién las hizo.

PREGUNTA 9 / 16

Perder la privada

Pregunta

¿Qué pasa si pierdes (o filtras) tu clave privada?

Respuesta razonada

Si la pierdes y no tienes copia, no hay forma matemática de recuperarla. Tu dinero queda atrapado en la blockchain para siempre — no hay banco que te emita otra. Si la filtras (la subes a un repo público, te la roban), quien la tenga puede firmar transacciones como tú y vaciar tu cuenta. Por eso existen hardware wallets, frases semilla (BIP-39) y custodios. La diferencia con la banca tradicional es radical: en el banco recuperas la tarjeta, en blockchain la pérdida es definitiva.

PREGUNTA 10 / 16

Hash no sirve para cifrar

Pregunta

¿Por qué no se puede usar simplemente un hash para hacer una clave pública con la que otros cifren?

Respuesta razonada

Porque un hash es una operación de una sola dirección pura: nadie puede deshacerla, ni siquiera tú. Para que el cifrado asimétrico funcione, la operación tiene que ser fácil de aplicar para cualquiera (con la pública) pero fácil de deshacer solo para quien tenga la privada. Eso requiere una función trampa (RSA con factorización de primos, ECC con curvas elípticas), no un hash. Si cifraras con un hash, ni el receptor podría descifrar — el hash sirve para integridad, no para cifrado.

PREGUNTA 11 / 16

Direcciones Bitcoin vs Ethereum

Pregunta

¿En qué se diferencian las direcciones de Bitcoin y de Ethereum?

Respuesta razonada

Bitcoin pasa la pública por SHA-256, luego por RIPEMD-160, añade versión + checksum y codifica en Base58Check, dando algo como '1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa'. Ethereum aplica Keccak-256 a la pública, coge los últimos 20 bytes y añade prefijo '0x', dando algo como '0xd35d2af...'. Bitcoin lleva doble hash y checksum integrado (robustez y menos errores al copiar); Ethereum es más simple pero sin checksum nativo (EIP-55 lo añadió después con mayúsculas y minúsculas).

PREGUNTA 12 / 16

Cliente-servidor vs P2P

Pregunta

¿Qué es una red P2P y cómo se diferencia del modelo cliente-servidor?

Respuesta razonada

En cliente-servidor hay dos roles fijos: el servidor presta el servicio y el cliente lo consume. Si cae el servidor, el servicio se cae. En P2P (peer-to-peer) no hay roles fijos: cada participante es a la vez cliente y servidor — pide recursos cuando los necesita y los sirve cuando se los piden. La red emerge de la unión de los peers; no hay punto central, no hay dueño único, y la caída de cualquier peer no rompe la red.

PREGUNTA 13 / 16

Caída de Napster

Pregunta

¿Por qué Napster se vino abajo cuando cerraron su servidor?

Respuesta razonada

Aunque Napster era P2P en los archivos (vivían en los ordenadores de los usuarios), tenía un punto centralizado: un servidor que mantenía el índice de quién tenía qué. Cuando buscabas una canción, ese servidor te decía qué peer la tenía. En 2001 la RIAA demandó y cerraron el servidor; sin él, los peers ya no podían encontrarse entre sí y la red entera murió. Lección que Bitcoin aprendió 8 años después: si hay UNA pieza centralizada, la red entera es vulnerable a ese punto.

BitTorrent

Pregunta

¿Qué innovación trajo BitTorrent respecto a redes P2P anteriores como Napster o eMule?

Respuesta razonada

Tres ideas combinadas. Primera: partir el archivo en trozos pequeños y descargarlos en paralelo de peers distintos (mucho más rápido). Segunda: cada peer sube los trozos que ya tiene mientras descarga los que le faltan, formando un swarm o enjambre. Tercera: incentivo tit-for-tat — quien más comparte, mejor velocidad obtiene. Resultado: una red P2P realmente eficiente, resiliente y con incentivos para colaborar. Sigue viva en 2026 distribuyendo Linux, juegos y todo tipo de contenido.

PREGUNTA 15 / 16

Pros y contras P2P

Pregunta

Cita dos ventajas y dos problemas típicos de las redes P2P.

Respuesta razonada

Ventajas: (a) Sin punto único de fallo — la red sigue funcionando aunque caigan peers; (b) Resistente a censura — no hay un servidor único que apagar para cerrarla. Problemas: (a) Descubrimiento: cuando entras a la red, ¿cómo encuentras a tus primeros peers? Típicamente con bootstrap nodes; (b) NAT y firewalls: la mayoría de peers están detrás de routers domésticos y no son directamente alcanzables — hay que resolver eso para que se puedan recibir conexiones.

PREGUNTA 16 / 16

Por qué blockchain necesita P2P

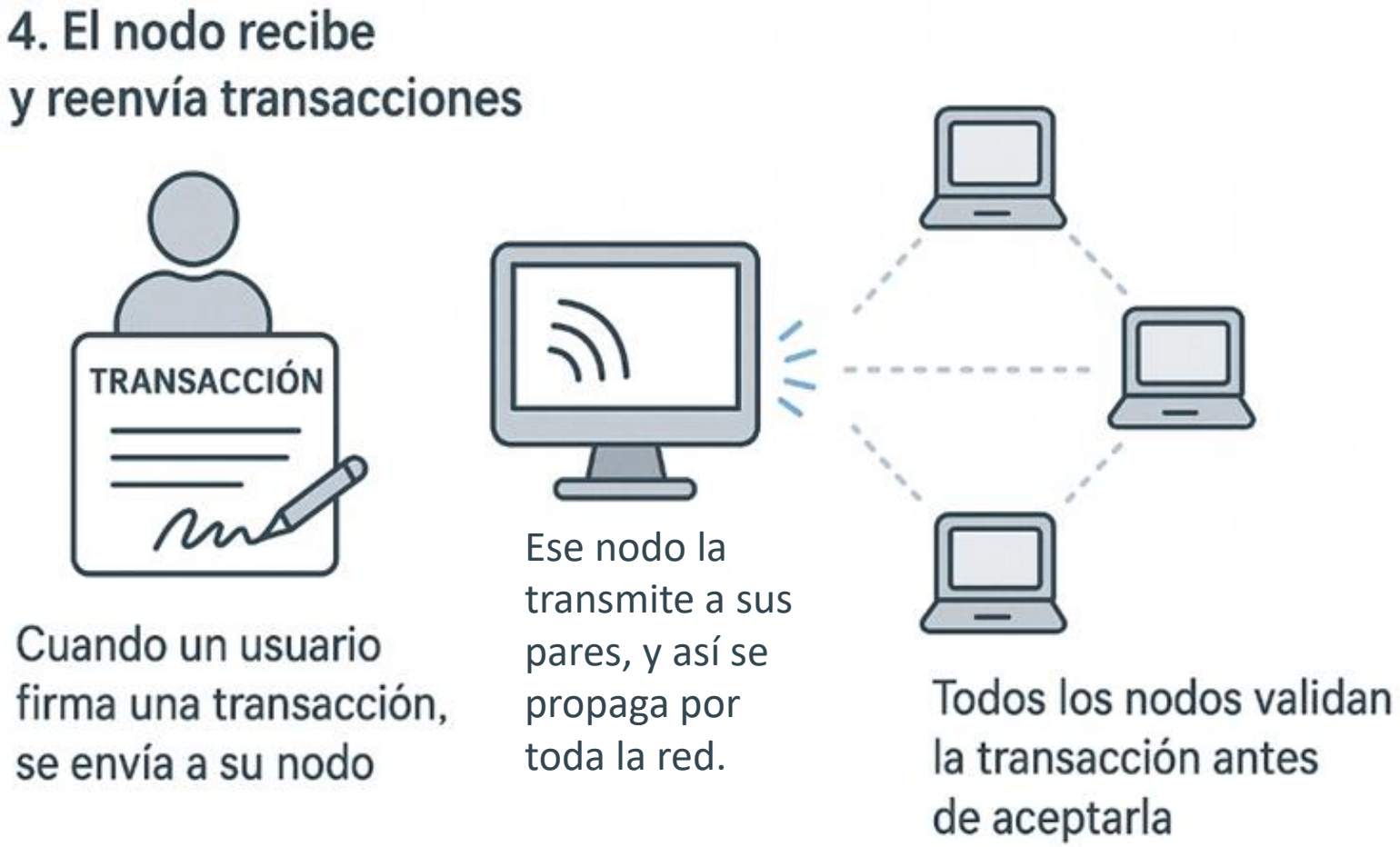
Pregunta

¿Por qué blockchain necesita una red P2P para funcionar?

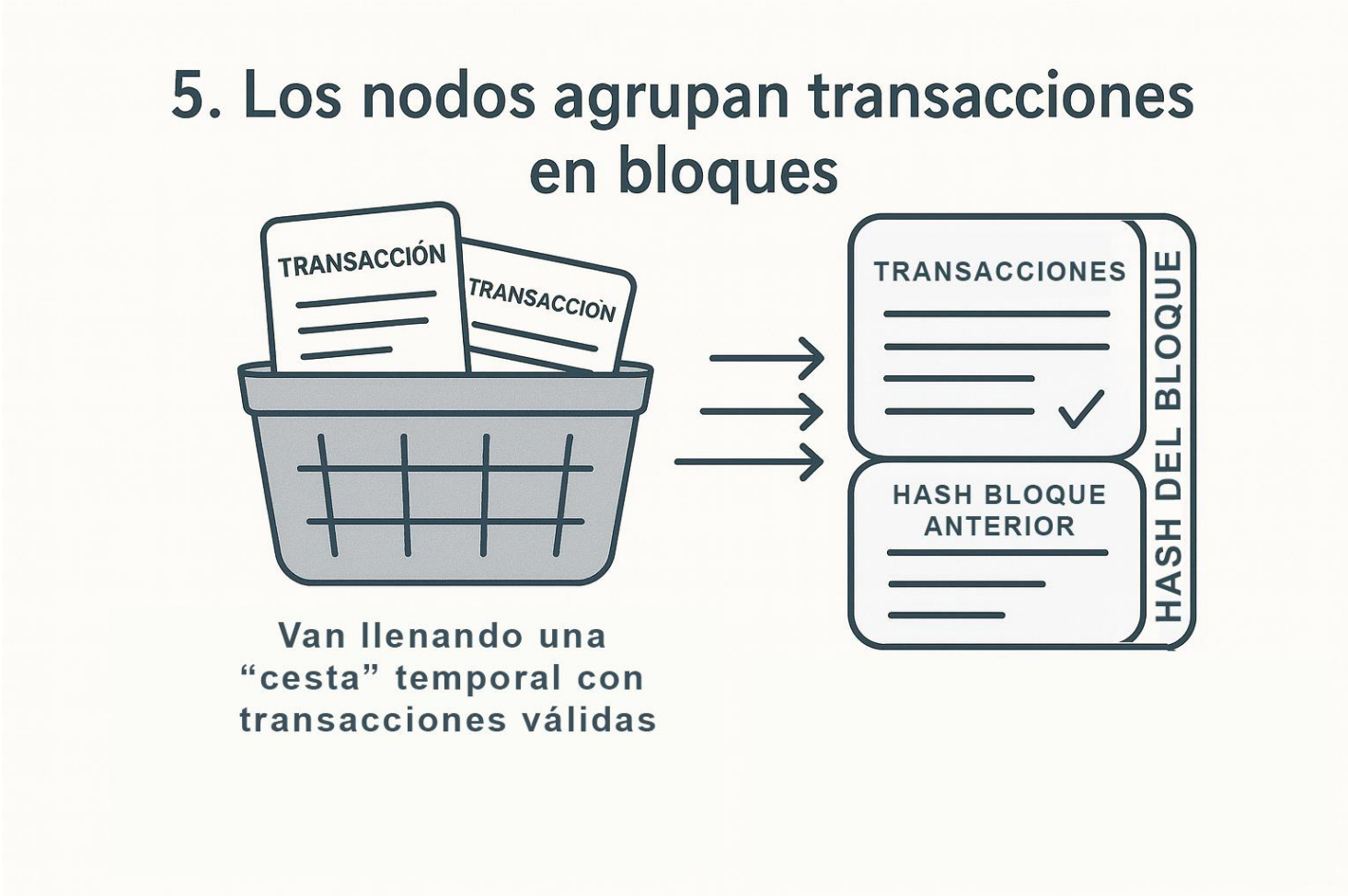
Respuesta razonada

Porque la promesa central de blockchain es eliminar al intermediario. Si tuvieras un servidor central llevando el libro de transacciones, ese servidor podría caerse, ser censurado, modificar registros o cobrar comisiones, exactamente lo que blockchain quiere evitar. La red P2P distribuye la responsabilidad entre todos los nodos: cada uno guarda una copia del libro, valida las transacciones y propaga los bloques. Sin red P2P no hay descentralización, y sin descentralización no hay blockchain real.

Blockchain – Red peer-to-peer



Blockchain – Red peer-to-peer



La cadena de bloques

Cómo los nodos reciben transacciones, las agrupan en bloques y los encadenan

Lo que ya sabemos

Concepto	Qué es	Para qué nos sirve
Hash	Huella digital de tamaño fijo (256 bits en SHA-256). Un solo bit cambia → hash totalmente distinto.	Integridad de los datos, identificadores únicos, encadenar bloques.
Clave pública / privada	Pareja de números matemáticamente ligados. La privada se mantiene en secreto, la pública se reparte.	Identidad digital, firmar transacciones, verificar que algo lo hiciste tú y no se ha manipulado.
Red P2P	Red sin servidor central donde cada nodo es a la vez cliente y servidor. Habla con sus vecinos por gossip.	Distribuir transacciones y bloques sin intermediarios, validar entre todos, sin punto único de fallo.

El viaje de una transacción

¿Qué pasa cuando Alicia envía 5 € a Bob?

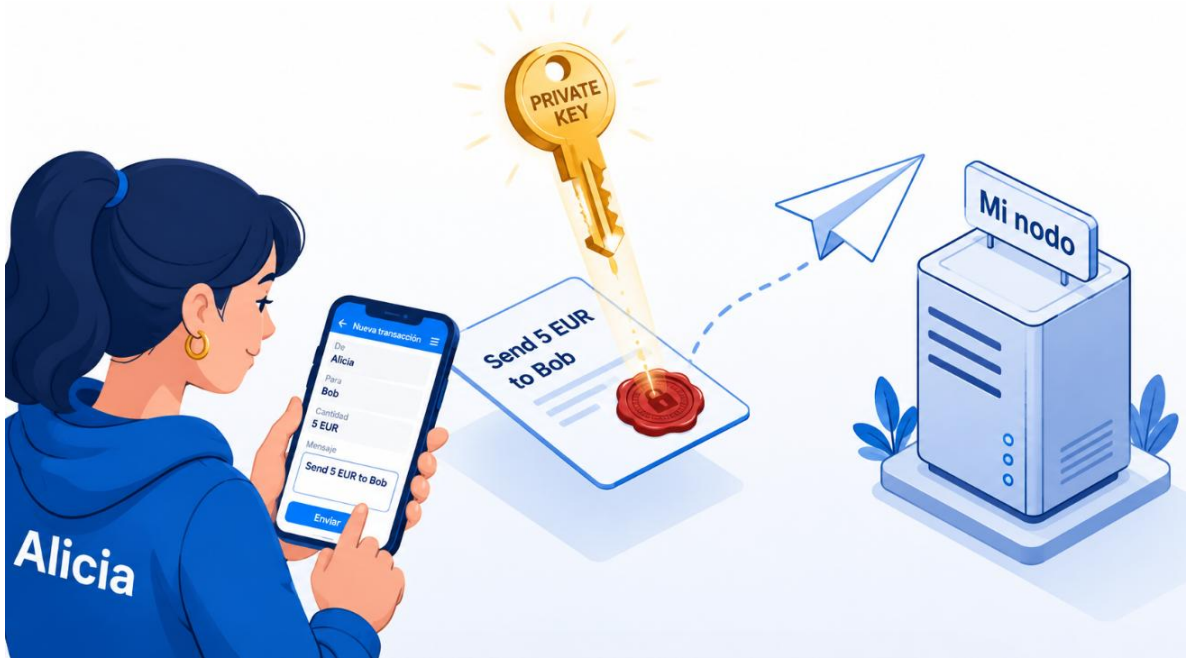
Paso 1 y 2 — qué ha pasado por dentro

Alicia compone la transacción: { de: Alicia, a: Bob, importe: 5 €, fecha: ahora }.

La firma con su clave privada → genera un código único (la firma) que solo ella puede producir.

La envía a su nodo más cercano de la red P2P. Ese nodo es la 'puerta de entrada' a la red.

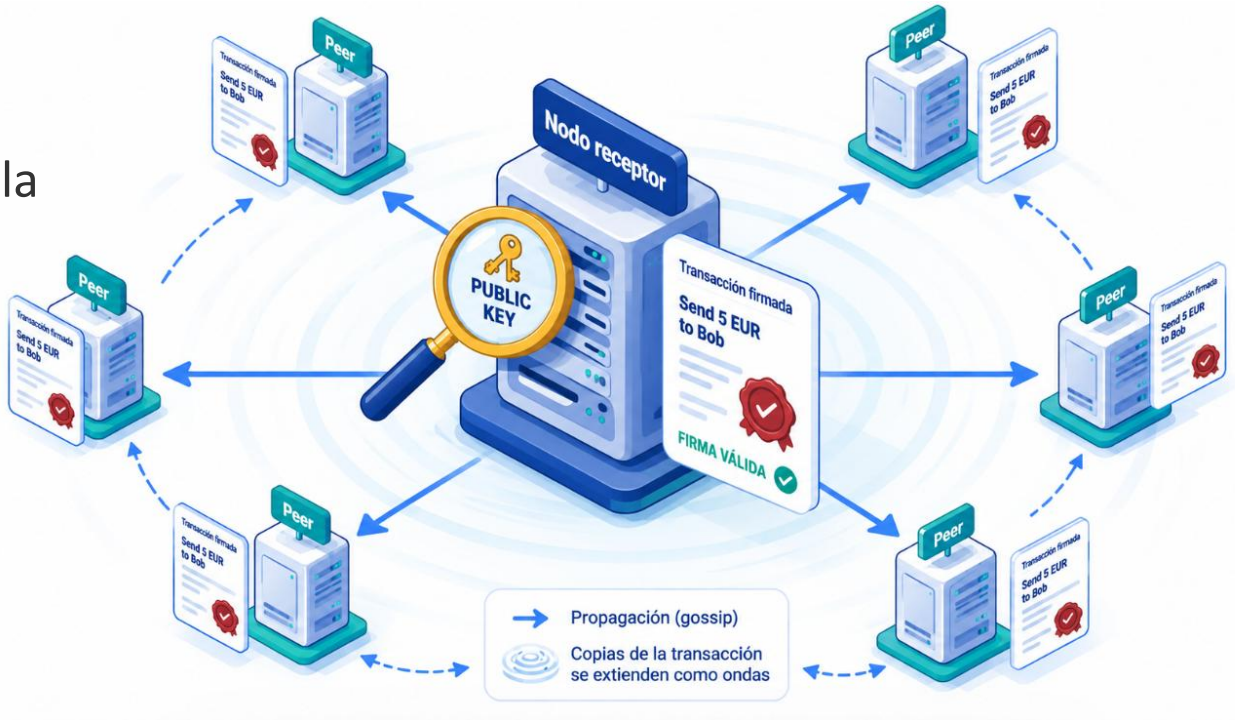
Lo importante: la transacción ya viaja con prueba de quién es el remitente. La privada nunca sale de Alicia, solo viaja la firma.



Paso 3 y 4 — qué ha pasado por dentro

Cualquier nodo que recibe la transacción puede verificarla:

- Coge la firma + el mensaje + la clave pública de Alicia.
- Si la firma cuadra → la transacción es auténtica (la hizo Alicia) y no se ha modificado.
- Si no cuadra → la descarta inmediatamente.

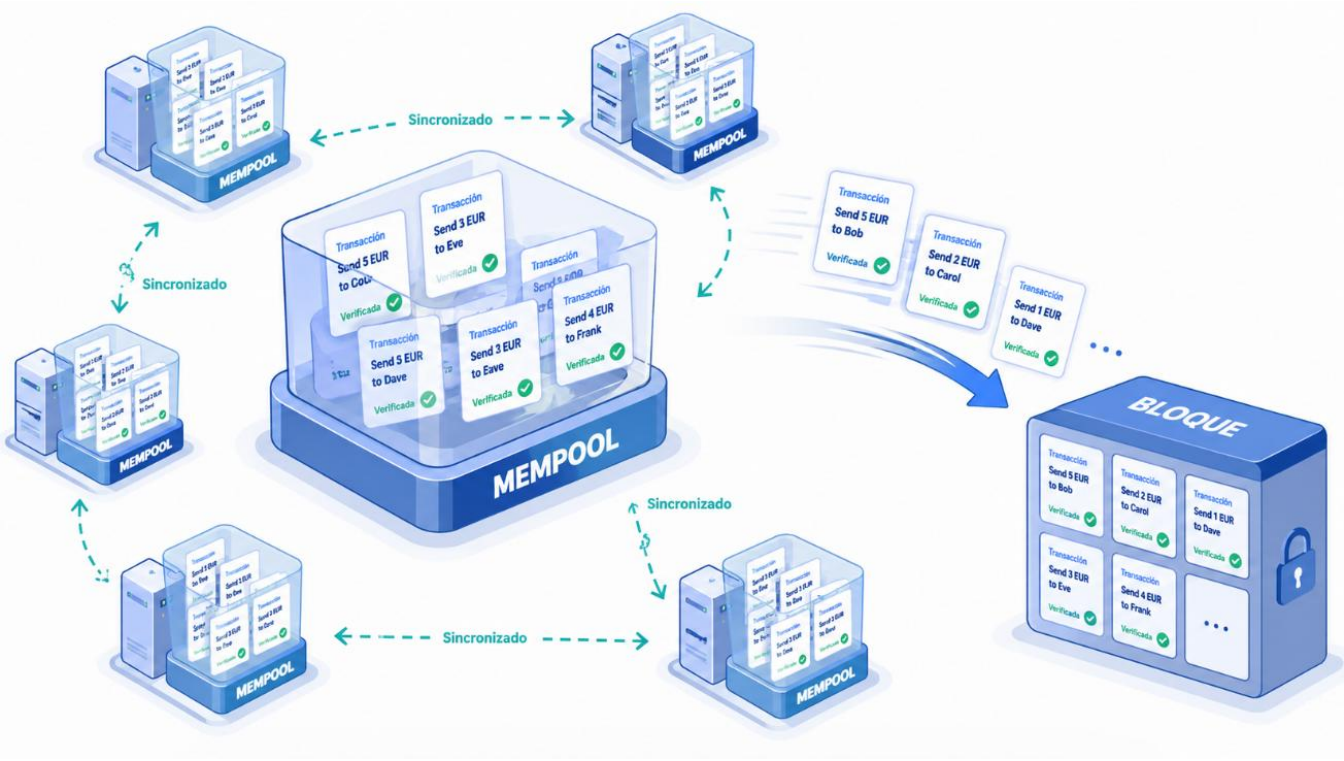


Si es válida, la propaga a sus vecinos (gossip). En segundos, toda la red conoce la transacción.

Idea clave: nadie tiene que confiar en quien le pasa la transacción. Cada nodo verifica por sí mismo.

El mempool — la sala de espera

Cada nodo guarda en su 'mempool' las transacciones validadas pero todavía no incluidas en ningún bloque. Es como una bandeja de entrada compartida entre todos.



Del mempool al bloque

Cómo se agrupan las transacciones y nace un bloque nuevo

¿Qué hay dentro de un bloque?

Cabecera (header) — la 'tarjeta de identidad' del bloque:

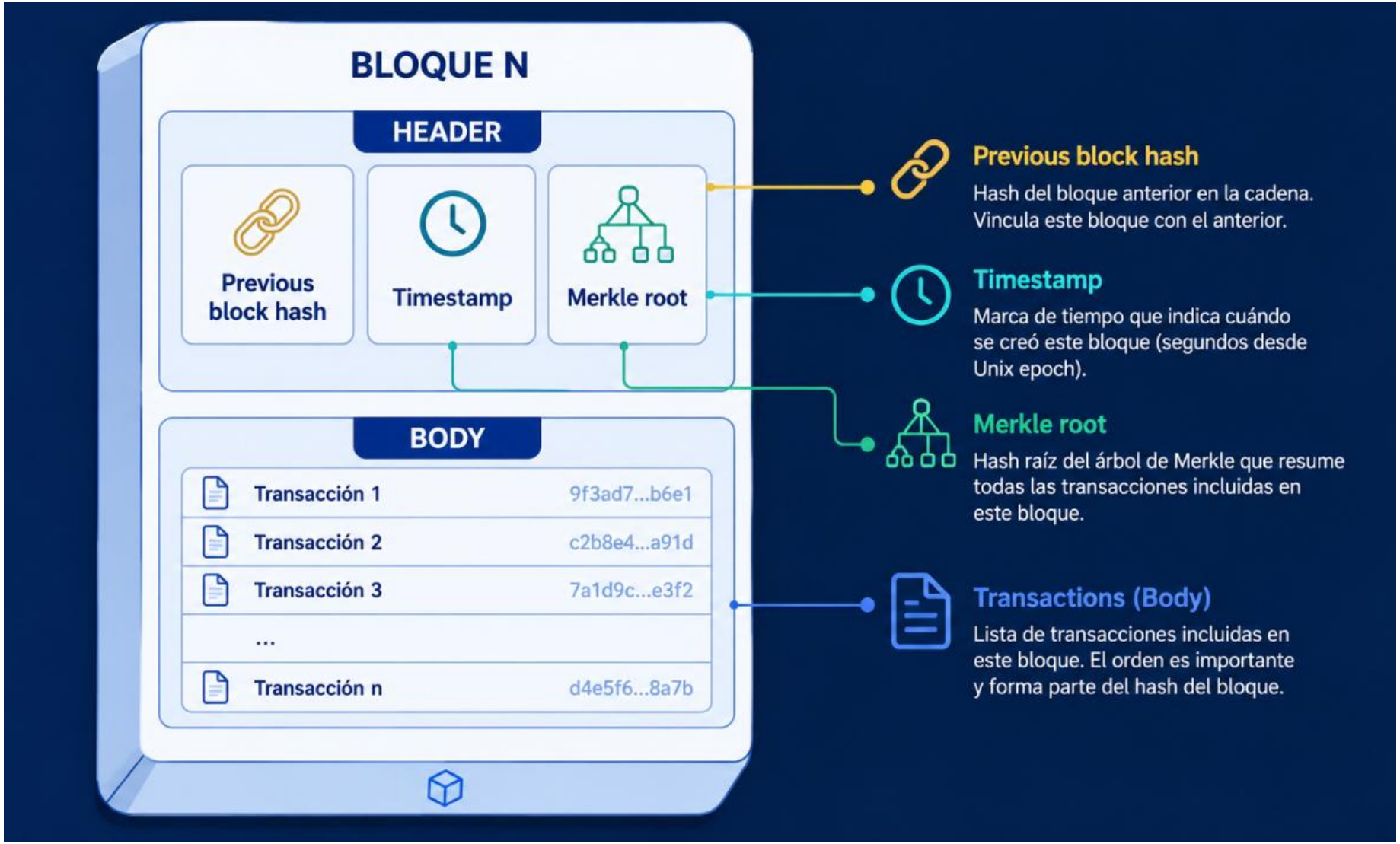
- Hash del bloque anterior (el eslabón hacia atrás).
- Marca de tiempo (cuándo se creó).
- Merkle root (resumen criptográfico de todas las transacciones del bloque).

Cuerpo (body) — los datos del bloque:

- La lista completa de transacciones que el bloque incluye.
- Pueden ser cientos o miles, según el tamaño máximo del bloque.

El identificador del bloque es el hash de su cabecera. Si cambias UN dato, cambia ese hash → es otro bloque.

¿Qué hay dentro de un bloque?



Cómo nace un bloque

Un nodo decide proponer un nuevo bloque. Pasos:

1. Mira su mempool y selecciona N transacciones (típicamente las que llevan más tiempo esperando).
2. Verifica cada una otra vez (por si hubo cambios).
3. Calcula la Merkle root de todas ellas.
4. Monta la cabecera: hash del último bloque conocido + timestamp + Merkle root.
5. Calcula el hash de la cabecera → ese es el identificador del bloque nuevo.
6. Lo propaga a sus vecinos como propuesta de bloque.

Otros nodos lo reciben, verifican y, si todo cuadra, lo añaden a SU copia de la cadena.

Por ahora damos por sentado que todos llegan a la misma conclusión. Luego veremos que esto NO es trivial.

La cadena

Por qué se llama 'cadena de bloques'

¿Por qué se llama cadena de bloques?

Cada bloque guarda dentro el hash del bloque anterior.

- El bloque N tiene 'prev_hash = hash(N-1)'.
- El bloque N+1 tiene 'prev_hash = hash(N)'.
- Y así sucesivamente, hasta el bloque 0 (génesis).

Eso es lo que crea la cadena: bloques unidos uno tras otro por su hash.

El bloque génesis es el primero. No tiene 'prev_hash' (no hay bloque anterior). Es el ancla criptográfica de toda la cadena.

Bloques encadenados por hash

La cascada de invalidación

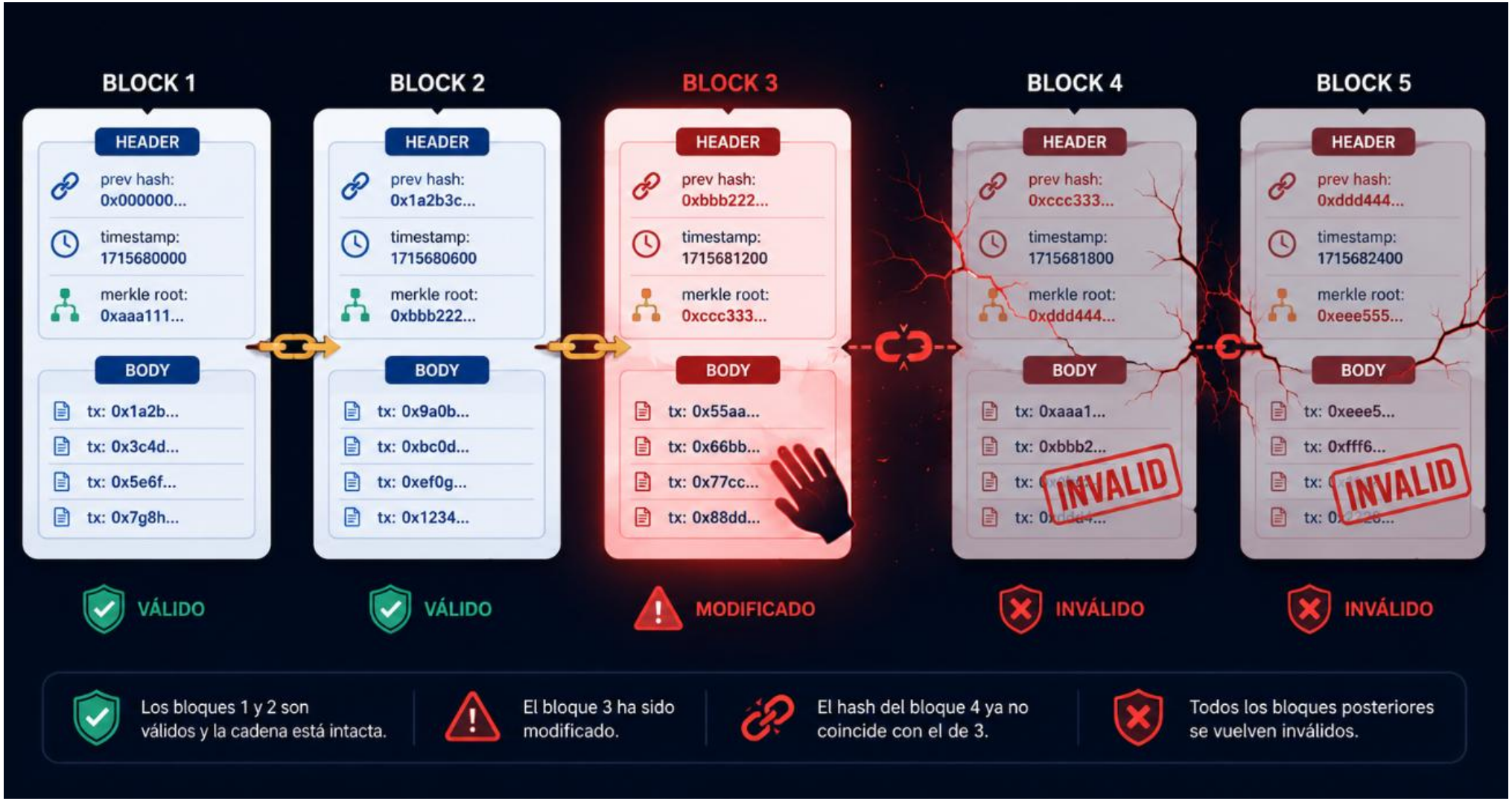
Imagina que un atacante quiere borrar una transacción del bloque N.

1. Cambia el dato → cambia el hash del bloque N.
2. Pero el bloque N+1 guardaba 'prev_hash = hash(N)'. Ahora ese campo apunta a un bloque que ya no existe.
3. La cadena se rompe en N+1.
4. Para que vuelva a ser coherente, el atacante tendría que reescribir N+1, N+2, ..., hasta el último bloque actual.

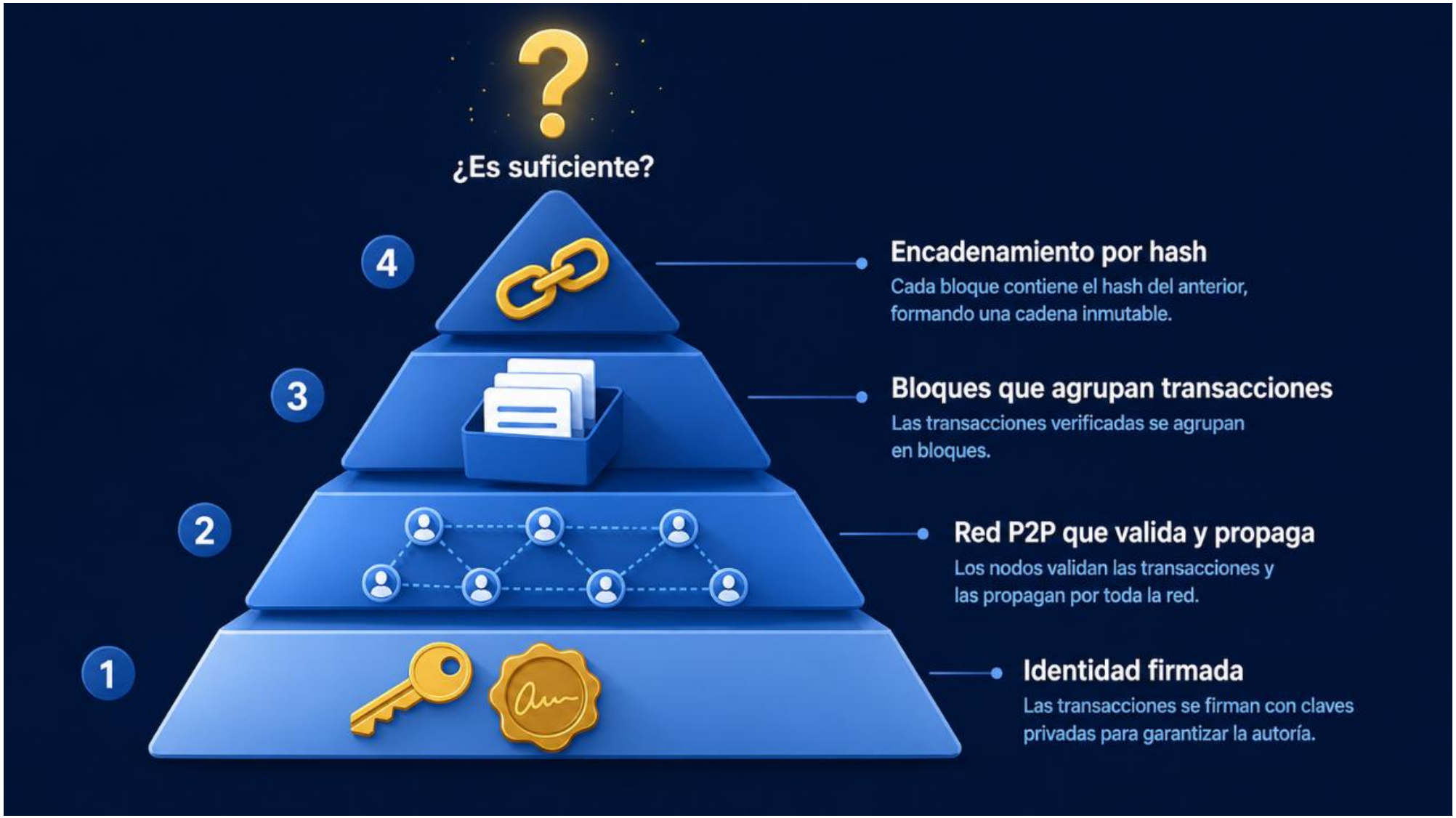
Es decir: para falsear el pasado, hay que rehacer TODA la historia posterior.

Esto **lo hace muy caro**, pero como veremos enseguida, no imposible. Es un punto crítico.

¿Qué pasa si alguien manipula un bloque del pasado?



Lo que ya tenemos



¿Es suficiente?

Vamos a hacer las preguntas incómodas

DEBATE

Las tres preguntas que rompen la idea

1. ¿Qué pasa si dos nodos crean bloques distintos al mismo tiempo?

2. ¿Qué impide que un atacante manipule SU copia, regenere todos los hashes y la imponga? Generar hashes es rápido…

3. ¿Cómo decidimos cuál es la 'verdadera' cadena si hay varias versiones circulando?

PREGUNTA 1

Dos bloques al mismo tiempo: el bifurcado

Respuesta intuitiva del alumno:

- Ambos bloques son válidos y traen transacciones bien firmadas.
- La red puede aceptar los dos. Eventualmente las transacciones acabarán incluidas en algún bloque.

Lo que pasa de verdad:

- Si la red acepta dos cadenas distintas, ya no hay un único ledger.
- Los saldos pueden divergir: el mismo dinero aparece gastado en cadenas distintas.
- Cada nodo podría ver una historia distinta — no hay consistencia global.
- Hace falta un mecanismo que decida cuál de los dos bloques sobrevive.

PREGUNTA 2

El atacante que rehace toda la cadena

Respuesta intuitiva del alumno:

- Las firmas impiden cambiar transacciones. Si modificas algo, la firma ya no cuadra.
- Por tanto, no se puede manipular el ledger.

Lo que pasa de verdad:

- Las firmas protegen el contenido de las transacciones, NO los hashes de los bloques.
- Un atacante puede dejar las transacciones tal cual y solo regenerar los hashes de los bloques.
- Calcular hashes es rápido y barato — millones por segundo en un PC normal.
- Si tiene recursos suficientes (muchos nodos), puede inundar la red con bloques fraudulentos coherentes.

Sin más defensas, el atacante con la mayor capacidad de cómputo gana.

PREGUNTA 3

¿Quién dice cuál es la cadena de verdad?

Respuesta intuitiva del alumno:

- Podemos votar entre nodos: la mayoría decide.
- Podemos confiar en la mayoría 'honesta'.
- Una marca temporal serviría para desempatar.

Lo que pasa de verdad:

- En un sistema sin autoridad central, no hay forma de saber quién es honesto.
- Cualquiera puede crear N nodos en su ordenador con identidades distintas.
- A esto se le llama ataque Sybil: un atacante crea miles de identidades falsas y gana cualquier votación.
- Las marcas temporales no funcionan: un atacante puede falsificarlas trivialmente.

No basta con 'votar' o 'confiar en la mayoría'. La votación misma es manipulable.

CONCLUSIÓN

Falta una pieza clave: el consenso

Tenemos transacciones firmadas, una red P2P que las propaga, bloques que las agrupan y un encadenamiento criptográfico que protege el pasado.

Pero TODAVÍA no tenemos cómo:

- Decidir qué bloque sobrevive cuando dos compiten al mismo tiempo.
- Impedir que un atacante con muchos recursos imponga su versión.
- Acordar entre nodos sin confianza previa cuál es 'la verdad'.

Eso es el mecanismo de consenso, y debe garantizar tres cosas:

1. Orden único de las transacciones.
2. Prevención del doble gasto (mismo dinero gastado en dos sitios).
3. Resistencia a ataques Sybil y a quien intente imponer una cadena alternativa.

Lo veremos a continuación

Mecanismos de consenso: cómo nodos sin confianza llegan a un acuerdo (Proof of Work, Proof of Stake, PBFT, Raft...)

Mecanismos de consenso

Cómo nodos sin confianza llegan a un acuerdo: del problema bizantino a Proof of Work y Proof of Stake

DÓNDE ESTAMOS

Lo que ya tenemos y lo que falta

Hasta ahora hemos visto:

- Hash: huella digital que protege la integridad de los datos.
- Claves pública/privada: identidad y firma de transacciones.
- Red P2P: nodos que se hablan entre ellos sin servidor central.
- Cadena de bloques: transacciones agrupadas en bloques encadenados por hash.

Y en la última sesión llegamos a la conclusión incómoda: TODO esto no basta.

- Dos nodos pueden generar bloques distintos a la vez (forks).
- Un atacante con recursos puede regenerar hashes y proponer una cadena alternativa.
- Sin autoridad central, votar es vulnerable a Sybil.

Hoy resolvemos esa pieza que falta: el mecanismo de consenso.

El problema del consenso

¿Cómo se ponen de acuerdo nodos que no se fían entre sí?

¿Qué es un mecanismo de consenso?

Un protocolo que permite a muchos nodos NO CONFIABLES acordar una única historia de transacciones.

Requisitos clave:

- Tolerar fallos: nodos que se caen o pierden mensajes.
- Tolerar nodos maliciosos: que mienten deliberadamente.
- Sin depender de una autoridad central.

La pregunta de fondo: ¿cómo fiarte de pares de los que no puedes fiarte? ¿Cómo llegar a un acuerdo con pares no fiables?

Lo lleva planteándose la informática 40 años. El planteamiento más famoso es un problema clásico: los generales bizantinos.

Los generales bizantinos: el escenario

El problema de los generales bizantinos

Varios generales rodean una ciudad y deben decidir entre dos opciones: atacar o retirarse.

Reglas del juego:

- Solo se comunican entre sí mediante mensajeros.
- Tanto generales como mensajeros pueden ser traidores y mentir.
- Hay un comandante que envía la orden inicial; los demás son tenientes que deben actuar coordinados.

Éxito: todos los generales LEALES toman la misma decisión.

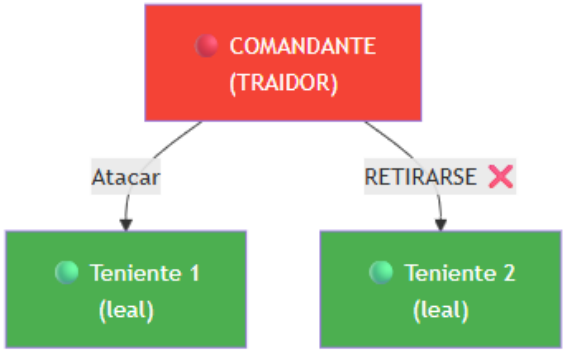
Fracaso: algunos leales atacan y otros se retiran. La ciudad sobrevive y los generales caen uno por uno.

El problema: ¿cuántos traidores como máximo puedes tener para que el consenso siga siendo posible?

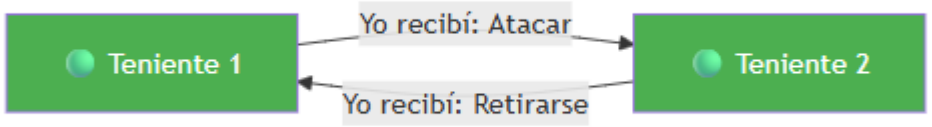
El problema de los generales bizantinos

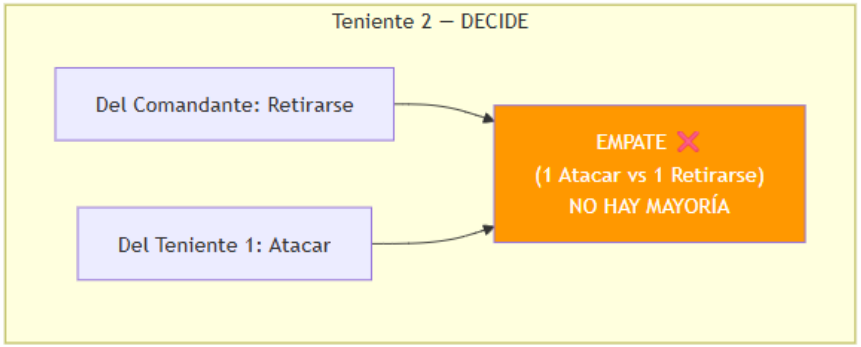
Con tres generales no hay solución

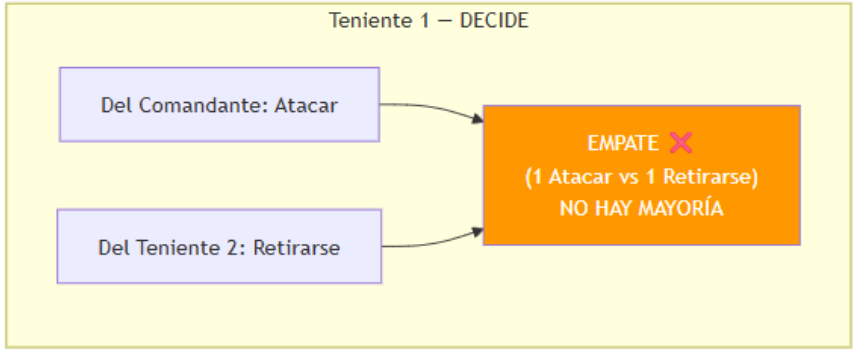
Ronda 1 — El Comandante (traidor) envía órdenes distintas



Ronda 2 — Los tenientes intercambian lo que recibieron (todos dicen la verdad)



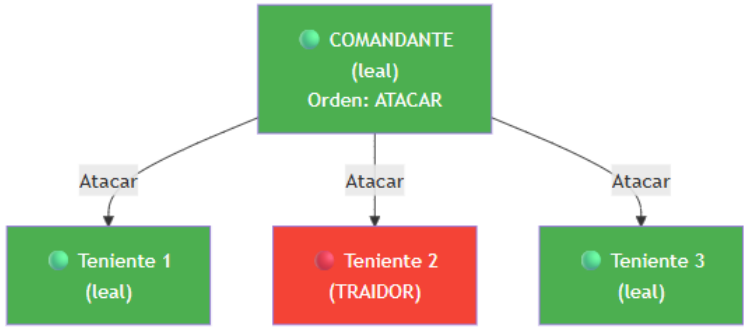




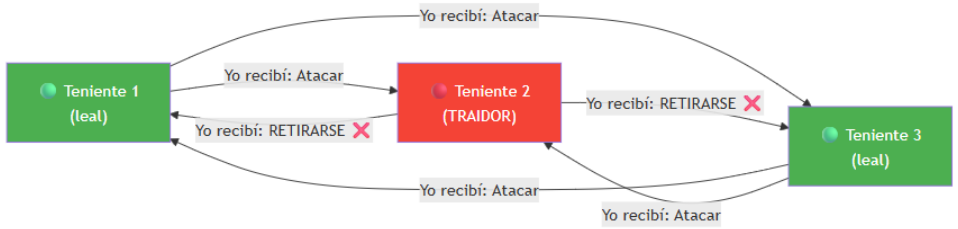
El problema de los generales bizantinos

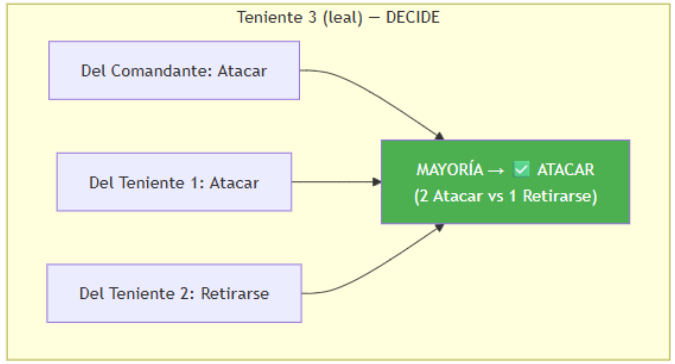
Con 4 generales y 1 traidor hay solución

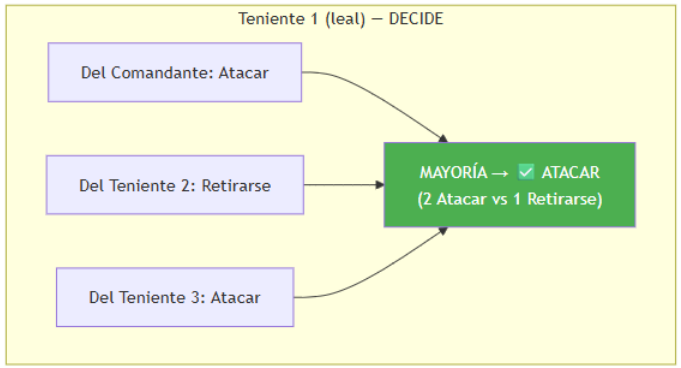
Ronda 1 — El Comandante envía órdenes



Ronda 2 — Los tenientes intercambian lo que recibieron (hay 1 traidor)



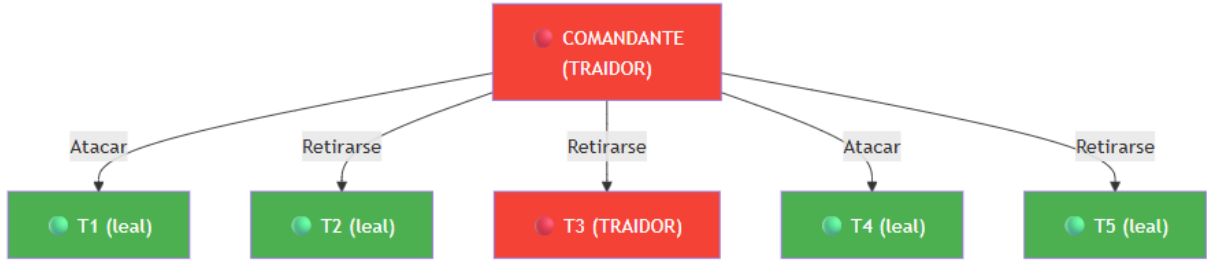




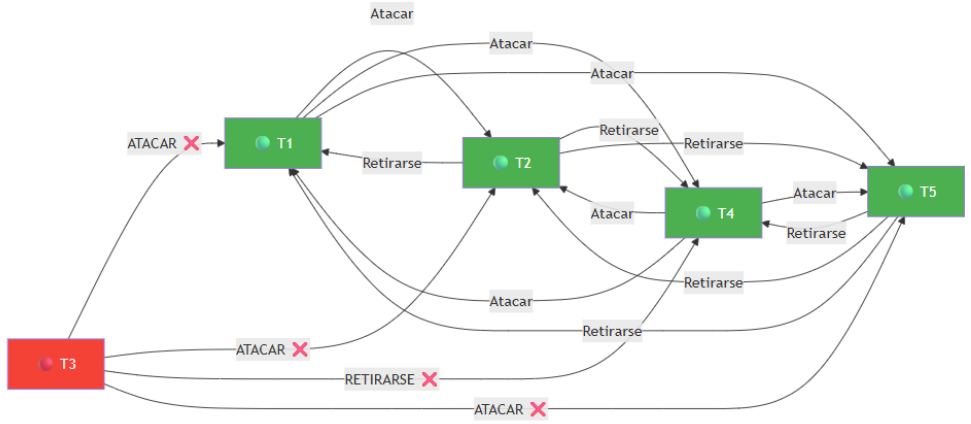
El problema de los generales bizantinos

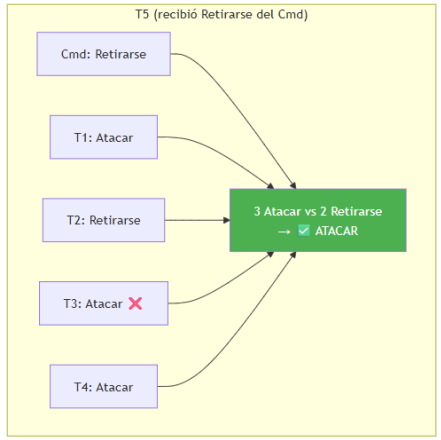
Con 6 generales y 2 traidores no hay solución

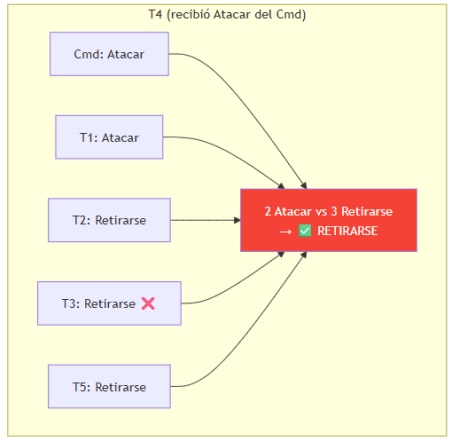
Ronda 1 — El Comandante envía órdenes distintas

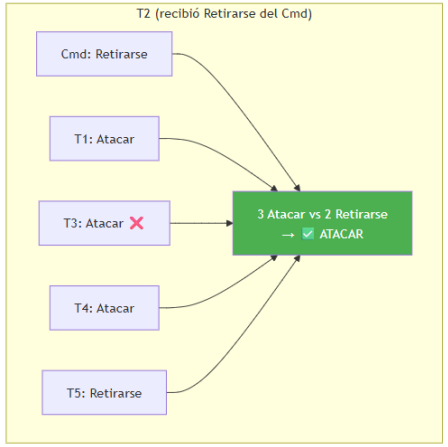


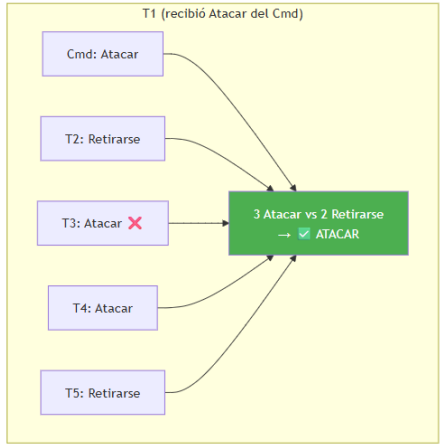
Ronda 2 — Los tenientes intercambian lo que recibieron (hay 1 traidor)











REGLA BFT

La regla general: necesitas más de 2/3 honestos

Lamport, Shostak y Pease demostraron en 1982 lo siguiente:

Para que N generales lleguen a consenso con F traidores, se necesita $N > 3F$.

- 3 generales, 1 traidor → $3 \leq 3 \cdot 1$ → IMPOSIBLE.
- 4 generales, 1 traidor → $4 > 3 \cdot 1$ → posible.
- 6 generales, 2 traidores → $6 \leq 3 \cdot 2$ → IMPOSIBLE.
- 7 generales, 2 traidores → $7 > 3 \cdot 2$ → posible.

Es decir: necesitas que más de 2/3 de los participantes sean honestos.

Es la base de los algoritmos llamados BFT (Byzantine Fault Tolerant). Hyperledger Fabric, Tendermint y muchos otros sistemas modernos los usan.

¿Qué tiene de bueno y qué de malo BFT clásico?

Lo bueno:

- Finalidad determinista: cuando se acuerda un bloque, ya está. No se reescribe.
- Funciona con muchos nodos identificados (Hyperledger Fabric, por ejemplo).

Lo malo:

- Necesita conocer de antemano la lista de participantes (identidades fijas).
- No protege contra ataques Sybil (un atacante crea N identidades falsas y consigue >1/3).
- El coste de mensajes crece muy rápido con el número de nodos: $O(N^2)$ o peor.

Resultado: BFT funciona muy bien en redes permisionadas (todos identificados) pero no encaja en una red abierta tipo Bitcoin, donde cualquiera puede entrar y nadie sabe quién es honesto.

Para redes abiertas, Satoshi Nakamoto propuso una idea radicalmente distinta.

La idea de Satoshi: Proof of Work

Hacer caro proponer bloques

La idea central de Proof of Work

Si no podemos confiar en las identidades, atémonos a algo que no se puede falsificar: el coste computacional.

Proponer un bloque tiene que ser DIFÍCIL.

- Tan difícil que requiera mucho cálculo (mucha electricidad).
- Tan ajustable que en promedio salga un bloque cada cierto tiempo (10 minutos en Bitcoin).
- Tan verificable que cualquiera puede comprobar en milisegundos si tu trabajo era válido.

Quien hace el trabajo (el minero) gana una recompensa por bloque válido.

- Honestidad como negocio: cumplir las reglas paga más que atacar.
- Atacar exige rehacer todo el trabajo computacional posterior. Económicamente inviable a escala.

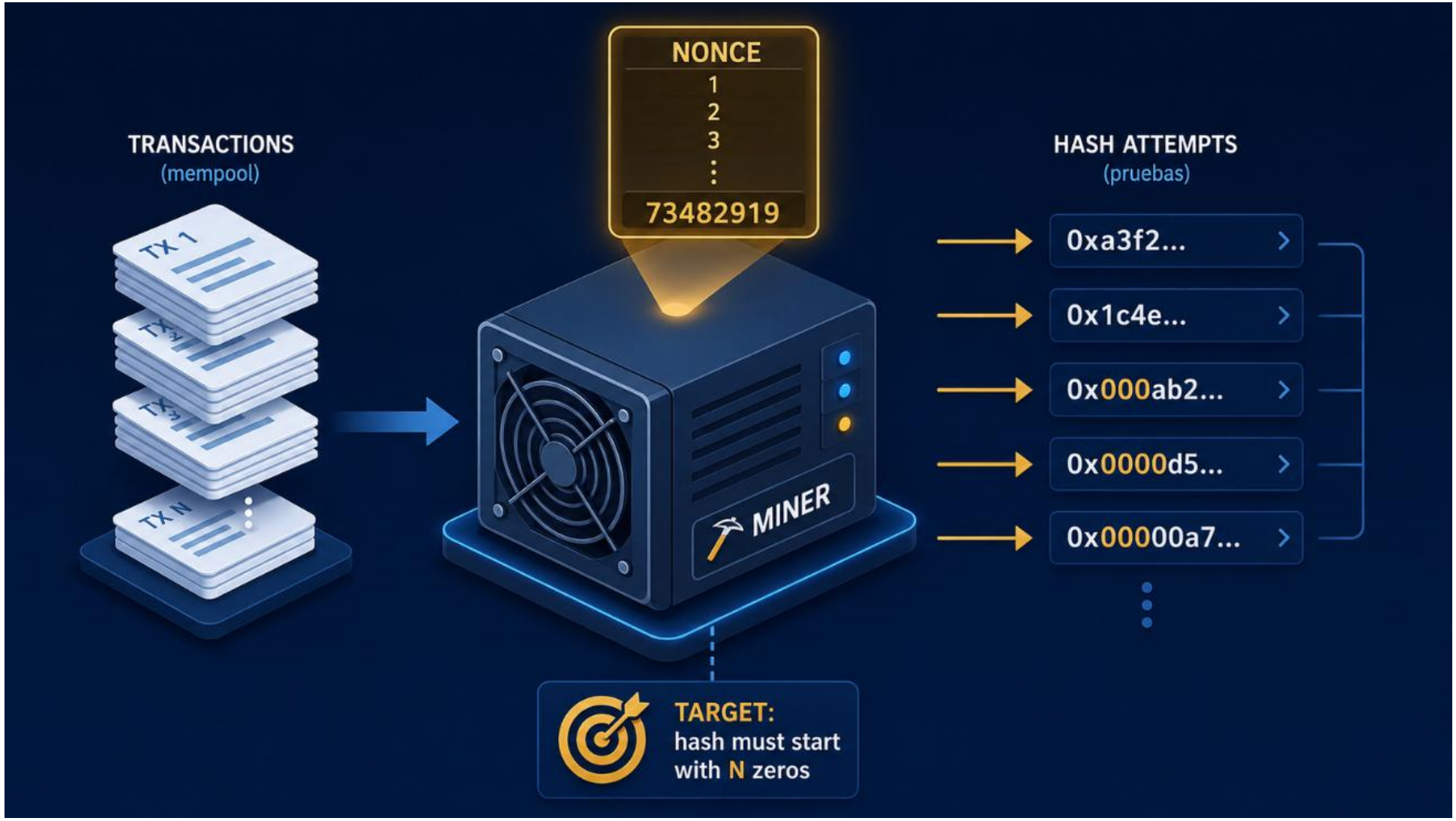
El resultado: finalidad PROBABILÍSTICA. Cuanto más viejo el bloque, más improbable revertirlo.

Proof of Work paso a paso

1. El minero recoge transacciones del mempool y arma un bloque candidato.
2. A la cabecera del bloque le añade:
   - El hash del bloque anterior (encadenamiento).
   - La marca de tiempo.
   - Un número arbitrario llamado nonce (puede valer cualquier cosa al principio).
3. Calcula el hash de la cabecera. ¿Empieza con suficientes ceros?
   - Si NO → cambia el nonce y vuelve a calcular.
   - Si SÍ → ¡bloque válido! Lo difunde a la red.
4. Cuando los demás reciben el bloque, verifican (instantáneo): hash correcto, transacciones válidas.
   - Si todo cuadra, lo añaden a su cadena. El minero recibe la recompensa (12,5 BTC, 6,25 BTC, 3,125 BTC...).
   - Si no, lo rechazan.

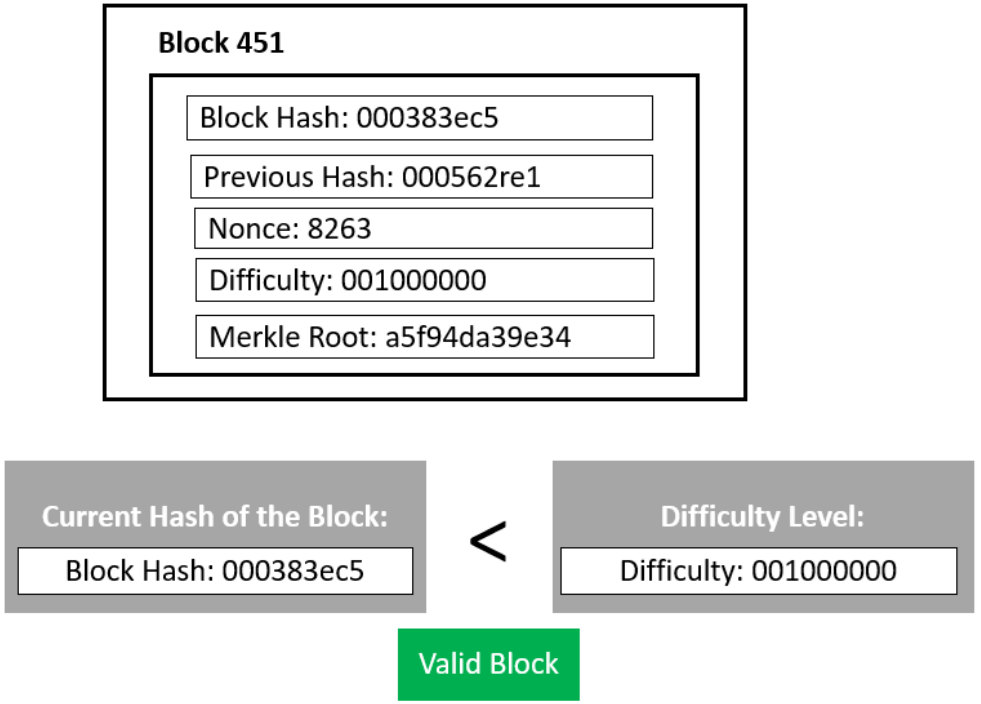
Lo asimétrico: encontrar un nonce válido cuesta MILLONES de intentos. Verificarlo cuesta UNO.

¿Qué hace un minero exactamente?

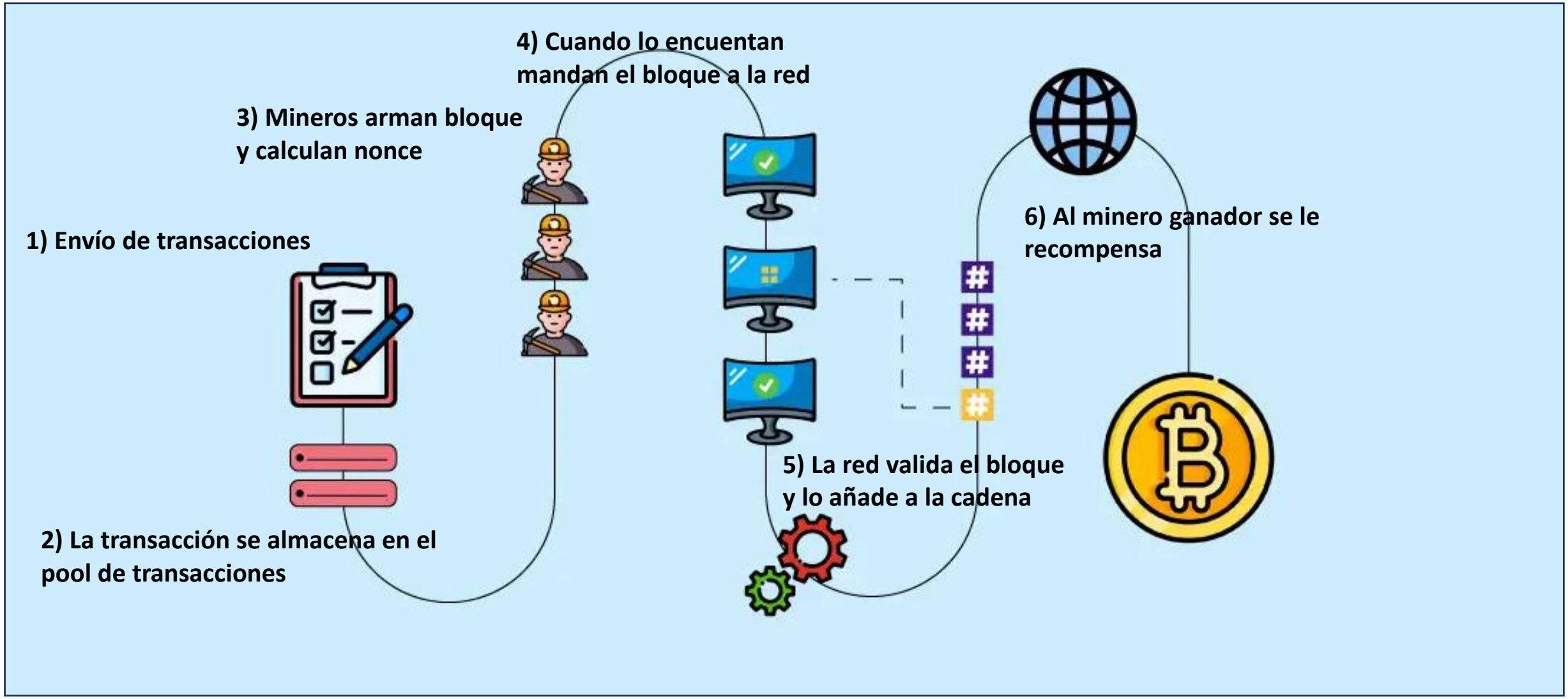


Proof of Work paso a paso

- Se basa en resolver un puzzle matemático a base de fuerza bruta de computación
- Minero recoge transacciones y arma un bloque candidato.
- A dicho bloque le añade el timestamp, el hash del bloque previo y un número arbitrario llamnado nonce.
- Minero varía el nonce y recalcula el hash del bloque.
- Repite millones de veces hasta que hash(block) < target.
- Si encuentra uno, lo difunde. Otros nodos verifican rápidamente (hash y transacciones).
- Si es válido, lo aceptan y se convierte en la punta de la cadena.



Proof of Work paso a paso



lake of shadows
S
TREASURE HUNT

Actividad – Encontrar hash con ceros al final

https://emn178.github.io/online-tools/sha256.html

- Ir a: https://emn178.github.io/online-tools/sha256.html
- Elegir el algoritmo que aparece por defecto (SHA256)
- Introducir texto aleatorio hasta obtener un hash que acabe con, al menos 3 ceros ‘000’

Actividad – Encontrar hash con ceros al final

Una solución es

Prueba de trabajo138

La regla de la cadena más larga

Pregunta abierta del bloque anterior: ¿qué pasa si dos nodos crean bloques al mismo tiempo?

En PoW puede ocurrir y de hecho ocurre. Dos mineros lejos uno de otro encuentran un nonce válido casi a la vez:

- Cada uno difunde su bloque.
- Los nodos cercanos a uno aceptan ese bloque; los cercanos al otro aceptan el otro.
- La red queda temporalmente bifurcada.

La regla de oro: la cadena con MÁS trabajo acumulado gana.

- Cuando uno de los dos lados encuentra el siguiente bloque (N+2), su cadena se hace más larga.
- Todos los nodos abandonan la cadena corta y adoptan la larga.
- Las transacciones de la cadena descartada vuelven al mempool.

Es probabilístico: con cada bloque adicional, la probabilidad de revertir un bloque pasado se desploma.

Las 3 preguntas incómodas, ahora contestadas

Problema	Cómo lo resuelve PoW
Doble gasto	Para reescribir una transacción del pasado hace falta rehacer el PoW de TODOS los bloques posteriores. Cada bloque exige una cantidad enorme de cómputo. Económicamente inviable salvo que controles >50% de la potencia de cómputo de la red.
Orden único	La regla 'cadena con más trabajo acumulado gana' es un criterio objetivo y verificable. Todos los nodos pueden calcularlo localmente sin votar. No hay ambigüedad.
Resistencia a ataques	Crear identidades falsas (Sybil) ya no sirve: lo que cuenta no son los nodos sino el cómputo. Un atacante necesita >50% del hashrate global, lo cual cuesta miles de millones en hardware y electricidad.
Incentivos	Mineros que cumplen las reglas reciben recompensas. Atacar cuesta más que cumplir, así que la mayoría se mantiene honesta por puro interés económico.

LÍMITE

El ataque del 51%

PoW no es invulnerable. Si alguien consigue reunir más del 50% del cómputo global de la red, puede:

- Generar bloques más rápido que el resto.
- Construir una cadena alternativa más larga que la honesta.
- Imponerla como la verdadera cuando la difunda.
- Reescribir transacciones recientes (doble gasto).

¿Es viable hoy?

- En Bitcoin (2026): exigiría inversión en hardware ASIC y electricidad por valor de varios miles de millones de dólares.
- Y el atacante, una vez gastado todo, derrumbaría la confianza en Bitcoin → su 'inversión' valdría cero.
- Por eso ningún actor racional lo intenta. La economía protege la red más que la criptografía.

Actividad – Blockchain humano

1. Pintar un ledger en la pizara con tres wallets con saldo
    1. Ana: 1000
    2. Nuria: 500
    3. Ramón: 2500
2. Organizar tres o cuatro grupos de mineros
3. El profesor hará las veces de usuario solicitando transacciones entre las wallets
4. Como algoritmo de PoW se podrá usar algún ardid matemático o semántico (por ejemplo encontrar una palabra que empiece por la letra del wallet origen, acabe por la del destino y contenga la primera leta de la cantidad trasnferida,

    P.ej: 20 de Nuria a Ramón => N….V….R (***navegar*** valdría)
5. Recompensar a los ganadores
6. Ver cuanto acumula cada grupo al final de la actividad

Mecanismos de consenso alternativos

Proof of stake

En lugar de gastar energía, los nodos deben bloquear monedas (“stake”).

Los validadores son elegidos de forma pseudoaleatoria proporcional a su stake.

Incentivo:

- Si actúan honestamente → recompensas.
- Si atacan o se equivocan → pierden parte del stake (slashing).

Mecanismos de consenso alternativos

Proof of Work	VS	Proof of Stake
Ganancia proporcional a la inversión en hardware y consumo eléctico		Ganancia proporcional a la cantidad de moneda bloqueada en el stake
Es más lento a la hora de producir bloques y consume electricidad		Más rápido produciendo bloques y a penas consume electrcidad
Lo hackers necesitan el 51% de la capacidad de computo		Lo hackers necesitan el 51% de las monedas de la red

Red peer-to-peer

https://geth.ethereum.org/

go-ethereum | DOWNLOADS | DOCUMENTATION | search CTRL K

Download go-ethereum

Bioelectric Infusers (v1.16.4)

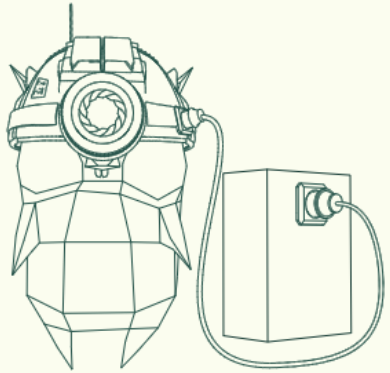
You can download the latest 64-bit stable release of Geth for our primary platforms below. Packages for all supported platforms, as well as develop builds, can be found further down the page. If you're looking to install Geth and/or associated tools via your favorite package manager, please check our installation guide.

FOR LINUX
GETH V1.16.4

FOR WINDOWS
GETH V1.16.4

FOR SOURCES
GETH V1.16.4

Release notes for Bioelectric Infusers (v1.16.4)



Práctica – Red peer-to-peer

1) Entrar en: https://github.com/jjnieto/Curso-HLFabric/blob/main/Modulo-1/practica_geth.md
2) Seguir los pasos de la práctica

CIERRE

Lo que hemos completado

Tenemos ya las cinco piezas que hacen viable una blockchain:

1. Hash → integridad y huellas digitales.
2. Claves pública/privada → identidad y firma.
3. Red P2P → distribuir sin intermediario.
4. Cadena de bloques → encadenamiento criptográfico del histórico.
5. Mecanismo de consenso → ponerse de acuerdo sin confiar.

Con esto podemos construir desde Bitcoin (PoW, abierto) hasta Hyperledger Fabric (BFT, permisionado).

El resto del curso es sobre cómo cada blockchain real combina y configura estas cinco piezas para resolver problemas concretos del mundo empresarial.